

Cap. 1

- 1.1. Não consiste apenas no código, inclui também a documentação associada, pois a Engenharia de Software abrange a especificação, desenvolvimento, validação e evolução de software, logo o produto final é um sistema completo suportado por documentação e processos de engenharia.
- 1.2. Software Genérico → Especificação realizada pela equipe de desenvolvimento.
Software Personalizado → Especificação realizada pelo próprio cliente.
Na prática significa que no software genérico os utilizadores não têm controlo direto sobre a especificação inicial nem na evolução do produto.
- 1.3.
 1. Fornecer as funcionalidades e desempenho pedidas pelo utilizador.
 2. Manutenção fácil.
 3. ^{Fabilidade} Dependência e Segurança.
 4. Aceitação.

Sugeridos:

 1. Eficiência (Não desperdiça recursos)
 2. Responsiveness (Tempo de Resposta)
 3. Verificabilidade (Capacidade de ser validada)
 4. Reutilização (Capacidade de voltar a ser utilizado noutros projetos)
- 1.4. Com o aumento do cloud computing e a distribuição massiva de sistemas, o consumo de energia dos centros de dados torna-se crítico, tornando a eficiência num caso imperativo, de modo a minimizar o impacto ambiental.
- 1.5. Não existem métodos universais para diferentes tipos de sistemas, os métodos dependem do tipo de aplicação a ser desenvolvida, dos requisitos do cliente e do background da equipa.
- 1.6.
 - Processo → Necessário para gestão e para que os sistemas sejam percebidos pela equipa.
 - Dependibilidade → É essencial que se possa confiar no software.
 - Gerenciamento de Requisitos → É crucial perceber e fazer uma gestão apropriada da especificação dos requisitos.
 - Reuso → Deve ser feito sempre que apropriado.

1.8. Todas envolvem responsabilidades que vão além dos conhecimentos técnicos, devem comportar-se de forma honesta e ética.

- Confidencialidade.
- Competência.
- Proteção da propriedade intelectual.
- Evitar a má utilização dos computadores.
- Gerir dilemas éticos.

→ Descreve o que são requisitos de utilizador e de sistema e de que forma é que estas estão relacionadas.

- Requisitos de Utilizador são descrições de alto nível que expressam o que o sistema deve fazer de uma forma compreensível para o cliente.
- Requisitos de Sistema detalham tecnicamente o que o sistema deve implementar para satisfazer os requisitos de utilizador.
- Ambos estão relacionados porque os requisitos de sistema derivam dos de utilizador.

→ Descreve o que são requisitos funcionais e não funcionais, justificando quais são mais importantes. Dá um exemplo de cada tipo.

- Requisitos Funcionais definem as funcionalidades específicas que o sistema deve oferecer.

Ex: Um sistema de e-commerce deve permitir que o utilizador adicione produtos ao carrinho.

- Requisito Não Funcionais são restrições de desempenho, segurança, usabilidade e requisitos de conformidade legal, que devem ser atendidas

Ex: O sistema deve carregar as páginas em menos de 2 segundos.

- Os Requisitos Funcionais são geralmente considerados mais importantes inicialmente, pois representam diretamente o valor para o utilizador. Porém, os Requisitos Não Funcionais são críticos para a qualidade e experiência de uso.

→ Indique, descrevendo, quais as atividades que devem fazer parte de qualquer tipo de processo de software, justificando a independência de cada tipo de atividade.

- As atividades essenciais são:
 - Especificação (definir requisitos e restrições)
 - Desenvolvimento (design e implementação)
 - Validação (assegurar conformidade com requisitos)
 - Evolução (adaptar o software a novas necessidades)
- Cada atividade é independente e essencial pois a Especificação define a base, o Desenvolvimento cria o software, a Validação assegura a qualidade e a Evolução permite adaptações futuras.

→ Descrevendo o que são cenários, qual o objetivo e o que devem incluir.

Indique também qual a relação que existe cenários e Use Cases.

- Cenários descrevem uma sequência específica de passos que um utilizador pode seguir para realizar uma tarefa, incluindo as várias interações com o sistema e potenciais variações.

- São utilizados para descrever situações concretas e são a base para os Use Cases, que agrupam cenários com um objetivo comum, como a compra de um produto.

→ Indique, justificando, quais as vantagens e desvantagens dos métodos de desenvolvimento que tenham por base uma entrega incremental.

- Vantagens: - Feedback Contínuo do Cliente:

A entrega incremental permite ao cliente experimentar funcionalidades parciais antes da conclusão do projeto, o que ajuda a reduzir o risco de entregas que não satisfaçam as expectativas finais.

- Flexibilidade para Ajustes e Mudanças:

Como o sistema é desenvolvido em partes, é mais fácil integrar novas necessidades ou corrigir erros, sem esperar pela conclusão do software.

- Menor Risco de Falhas Críticas:

Com entregas frequentes, falhas ou problemas são identificados e corrigidos rapidamente, reduzindo a probabilidade de erros graves.

- Desvantagens: - Degradação da Estrutura do Software:

Com várias iterações e mudanças, a qualidade do código pode diminuir, tornando-o mais complexo e difícil de manter.

- Maior Custo de Manutenção e Refatoração:

As mudanças contínuas podem exigir constante refatoração do código, aumentando custos de manutenção e pode resultar em mais tempo investido em corrigir ou melhorar código antigo.

→ Indique, justificando, qual o principal problema de processos de software baseados em entrega incremental e como este problema pode ser minimizado.

- O principal problema dos processos incrementais é a possível degradação da estrutura do software com a adição de funcionalidades.
- Para mitigar o problema devemos realizar boas práticas de codificação, documentando e reverendo o código regularmente.

→ Diagrama Use Cases.

- Em UML, o diagrama de Use Cases ilustra os principais atores e casos de uso, mostrando como os utilizadores interagem com o sistema.
- Inclui associações como «include» para fluxos comuns e «extend» para variações.

→ Descrição textual Use Cases de...

Ex: Encomendar uma Pizza

- Ator Principal: Cliente
- Passos: Cliente faz login → Escolhe a opção Encomendar →
→ Seleciona produtos → Visualiza o preço total →
→ Confirma a encomenda e fornece dados de pagamento.
- Extensões: Se o código de produto não existe, o sistema retorna ao passo de seleção.

→ Indique porque é que o processo de desenvolvimento profissional de software deve sempre usar as técnicas e conceitos associados à Engenharia de Software, indicando as suas vantagens da sua utilização, e os perigos da sua não utilização.

- Deve ser sempre usado porque esta é uma disciplina específica preocupada com todos os aspetos da produção de software.
- Vantagens: Software mais confiável, manutenção fácil e menos falhas.

- Perigos De Não Usar: Atrasos, custos altos, falhas e software difícil de manter.

→ Atributos que contribuem para a qualidade do software.

- Eficiência - Não desperdiçar recursos.
- Manutenção fácil - Deve ser permitida a evolução do software, pois é algo inevitável.
- Dependência e Segurança - Deve ser fiável e seguro, em caso de falha não pode danificar fisicamente ou economicamente o sistema, utilizadores com más intenções não devem conseguir causar danos.
- Aceitação - Deve ser aceite e utilizável por todos os utilizadores para o qual foi desenvolvido.

→ Principais atividades da Engenharia de Software.

- Especificação - Definir o que o sistema deve fazer, serviços, funcionalidades e restrições do sistema.
- Desenvolvimento - Converter a especificação em software útil através de desenho e implementação.
- Aceitação - Verificar que o software cumpre as especificações e requisitos de cliente.
- Evolução - Alterar e modificar o software de acordo com os requisitos.

→ Os 2 tipos de desenvolvimento de software principais.

- Modelo Waterfall - É baseado em planos, onde cada etapa só se inicia quando a anterior terminar, as etapas de especificação e desenvolvimento são distintas e separadas.

Vantagens: Simples e bem estruturado.

Desvantagens: Não é possível voltar atrás e é difícil lidar com novas restrições.

- Desenvolvimento Incremental - As etapas de especificação, desenvolvimento e validação ocorrem simultaneamente.

Vantagens: Entrega do software mais rápida, feedback contínuo e custo reduzido para induzir alterações.